Final Project - Profile-Based Cache Bypassing

1. Introduction

In the final project, you will extend your PA3 cache bypassing assignment, and model a profile-based cache bypassing logic in Accel-sim. You can see the original ICS'15 paper here. For this project, we will use Accel-sim, which is the modernized version of Gpgpu-sim. We created a Docker image with all the dependencies installed. Run the command below to download it. Beware that the Docker image size is quite large at ~17GB due to the dependencies.

```
$ docker pull ystozlu/ece786-accelsim:latest
```

Then launch a container,

```
$ docker run -it ystozlu/ece786-accelsim /bin/bash
```

2. Task Description

You have 2 subtasks that you need to complete in order.

2.1. Cache Efficiency Analysis

The ICS paper aims at accelerating cache-unfriendly benchmarks. Your first task is to identify the categories of benchmarks. There are three categories,

1. **Cache Unfriendly:** L1 cache bypassing improves the IPC(Instruction-per-cycle)
2. **Cache Insensitive:** The IPCs change little when bypass logic is enabled/disabled
3. **Cache Friendly:** L1 cache bypassing leads to lower IPC

You can run the benchmarks by enabling and disabling the "`-gmem_skip_L1D`" option in gpgpusim.config. This option enables/disables L1 data cache bypass logic for global memory access. As a rule of thumb, if you see at least 3-4% improvement in IPC when you enable bypassing, you can consider that benchmark cache unfriendly.

The configuration you should use for both tasks is ECE786_NCSU, which is a modified version of the SM2_GTX480 configuration we created for the class. You can identify the categories of benchmarks by comparing the IPCs of simulation with and without cache bypass enabled. The config also caps the maximum number of simulated instructions to 1M, so simulations should not take very long.

The benchmark traces you need to use are included in the Docker image. Namely, **BFS**, **LPS** and **NQU** from ISPASS; **BP**, **HS**, and **LUD** from Rodinia. Therefore, you need to run a total of 6 benchmarks. You can use the following command inside the container to run a simulation,

```
$ cd /accel-sim/accel-sim-framework
$ # Run this when you first enter the container
$ source gpu-simulator/setup_environment.sh

$ gpu-simulator/bin/release/accel-sim.out -trace
./util/tracer_nvbit/LPS_traces/kernelslist.g -config
./gpu-simulator/gpgpu-sim/configs/tested-cfgs/ECE786_NCSU/gpgpusim.co
nfig -config
./gpu-simulator/configs/tested-cfgs/SM86_RTX3070/trace.config
```

You can change the `-trace` argument to run other benchmarks. You can find the actual simulator code under `/accel-sim/accel-sim-framework/gpu-simulator/gpgpu-sim`.

When you make changes to the simulator code, you can simply run `make` to rebuild it.

2.2. Benchmark Performance Study (Timing Simulator)

In the second task, you are required to implement the profile-based cache bypass logic. The process can be broken down into 2 steps,

1. In the first run of the simulation, profile the reference frequency of each data block in the L1 data cache. You should create a per-SM profiler to record the intra-SM data reference frequency. For per-SM profiling, you need to have a data structure for each SM and save the reference counters in the following format,

   ```
   < SM 0, kernel 1 : addr1 ref, addr2 ref, ...addrn ref
   kernel 2 : addr1 ref, addr2 ref, ...addrn ref
   ...
   kernel n : addr1 ref, addr2 ref, ...addrn ref >
   ...
   < SM n, kernel 1 : addr1 ref, addr2 ref, ...addrn ref
   kernel 2 : addr1 ref, addr2 ref, ...addrn ref
   ...
   kernel n : addr1 ref, addr2 ref, ...addrn ref >
   ```

   After profiling, you need to dump out your counters into a file(s) for later use.

2. In the second run of the simulation, you need to first load your per-SM profile data into the simulator before the simulation starts, then use the counters to implement your cache bypass logic. The cache bypass logic is as follows: **For a request with a reference counter less than 3, bypass the L1 data cache**. Use the **cache unfriendly** benchmarks for this task.

3. Report

Prepare a brief report with the following content.

For the first task, you need to report the results of the benchmarks. Create a table to show the results of cache unfriendly, cache insensitive and cache friendly benchmarks. Your table should have the following format,

Benchmark name	kernel_name	kernel_launch_uid	IPC without cache bypassing	IPC with cache bypassing	Percent change in IPC	Benchmark category

For the second task, briefly explain your implementation of the profile-based cache bypass. Create another table to show the results with profile-based bypassing. Include detailed commands on how to generate and use the profiling data. (E.g. “comment out this code, enable that config option, rebuild the simulator, run the benchmarks” etc.)

4. Submission

- Create a directory named ”task1”, copy the simulator output files you generated for task 1 to this directory.
- Create a directory named ”task2”, copy the files you modified for task 2, your profile data, and the output files you generated for task 2 to this directory.

Zip ”task1”, ”task2” and your report as <unityId.zip>.

```
$ zip -r <unityId>.zip task1 task2 report.pdf
```

5. Cheating and LLMs

- You are supposed to do this assignment on your own. You can have discussions with your classmates, but we expect you to write your own code at the end of the day.
- Do not submit AI generated code/report.

Violation of these rules may result in zero marks and disciplinary action.